Kalman Filter Theory and Applications Equation Drilldown

https://github.com/musicarroll/kalman_course

Michael L. Carroll

August 2, 2023

©2023 by Michael L. Carroll

Part I

The Five Basic Kalman Equations

Topics

- Understanding the Equations: Heuristic Introduction

- Understanding the Equations: Heuristic Introduction
- **Equation Drilldown: Taking the Equations Apart**

- Understanding the Equations: Heuristic Introduction
- **Equation Drilldown: Taking the Equations Apart**
- State Space Concepts

- Understanding the Equations: Heuristic Introduction
- **Equation Drilldown: Taking the Equations Apart**
- State Space Concepts

State Space Concepts

Topics

State Space Concepts

Topics

- Dynamic Systems: Differential Equations and the Concept of State Space

State Space Concepts

Topics

- Dynamic Systems: Differential Equations and the Concept of State Space
- Diagrams and State Space

State Space Concepts

Topics

- Dynamic Systems: Differential Equations and the Concept of State Space
- Diagrams and State Space
- State Transition Matrix

State Space Concepts

Topics

- Dynamic Systems: Differential Equations and the Concept of State Space
- Diagrams and State Space
- State Transition Matrix

Dynamic Systems

Dynamic Systems

- Time-dependent behavior of physical systems often modeled by differential equations

Dynamic Systems

- Time-dependent behavior of physical systems often modeled by differential equations
- Called dynamic systems or dynamical systems

Dynamic Systems

- Time-dependent behavior of physical systems often modeled by differential equations
- Called dynamic systems or dynamical systems

Dynamic Systems

Dynamic Systems

- In Newton's law $F = ma$, the resultant force F acts on mass m to produce acceleration $a = \dfrac{dv}{dt}$

Dynamic Systems

- In Newton's law $F = ma$, the resultant force F acts on mass m to produce acceleration $a = \dfrac{dv}{dt}$
- Resultant force is vector sum of all forces acting on the body, both internal coupling forces and external input forces

Dynamic Systems

- In Newton's law $F = ma$, the resultant force F acts on mass m to produce acceleration $a = \dfrac{dv}{dt}$
- Resultant force is vector sum of all forces acting on the body, both internal coupling forces and external input forces
- Restoring forces (such as Hook's law $-ky$ force in simple oscillator problem) are often considered internal to system

Dynamic Systems

- In Newton's law $F = ma$, the resultant force F acts on mass m to produce acceleration $a = \dfrac{dv}{dt}$
- Resultant force is vector sum of all forces acting on the body, both internal coupling forces and external input forces
- Restoring forces (such as Hook's law $-ky$ force in simple oscillator problem) are often considered internal to system
- Internal forces not considered as an input *per se*

Dynamic Systems

- In Newton's law $F = ma$, the resultant force F acts on mass m to produce acceleration $a = \dfrac{dv}{dt}$
- Resultant force is vector sum of all forces acting on the body, both internal coupling forces and external input forces
- Restoring forces (such as Hook's law $-ky$ force in simple oscillator problem) are often considered internal to system
- Internal forces not considered as an input *per se*

Dynamic Systems

Dynamic Systems

- Response or output of (mechanical) system is motion, accelerated motion given by a

Dynamic Systems

- Response or output of (mechanical) system is motion, accelerated motion given by a
- Simple harmonic oscillator is dynamic system, one in which there is one internal force determining system behavior: $F = -ky$
  (more complicated oscillators may also include frictional forces and internal)

Dynamic Systems

- Response or output of (mechanical) system is motion, accelerated motion given by a
- Simple harmonic oscillator is dynamic system, one in which there is one internal force determining system behavior: $F = -ky$
  (more complicated oscillators may also include frictional forces and internal)
- External forces (i.e., a time-dependent input functions) may also be present, e.g.:

$$\frac{d^2y}{dt^2} + \frac{k}{m}y = \sin \omega t$$

Dynamic Systems

- Response or output of (mechanical) system is motion, accelerated motion given by a
- Simple harmonic oscillator is dynamic system, one in which there is one internal force determining system behavior: $F = -ky$
  (more complicated oscillators may also include frictional forces and internal)
- External forces (i.e., a time-dependent input functions) may also be present, e.g.:

$$\frac{d^2y}{dt^2} + \frac{k}{m}y = \sin \omega t$$

Dynamic Systems

Dynamic Systems

- Most general second-order equation for mechanical system of this type (according to Newton's law) is:

$$\frac{d^2y}{dt^2} = \frac{1}{m}F(y, \dot{y}, t) \tag{1}$$

Dynamic Systems

- Most general second-order equation for mechanical system of this type (according to Newton's law) is:

$$\frac{d^2y}{dt^2} = \frac{1}{m}F(y, \dot{y}, t) \tag{1}$$

- If we can express forcing function $F(y, \dot{y}, t)$ as sum of two functions $F_1(y, \dot{y})$ and $F_2(t)$, then we can rearrange into input-output form:

$$\frac{d^2y}{dt^2} - \frac{1}{m}F_1(y, \dot{y}) = \frac{1}{m}F_2(t)$$

Dynamic Systems

- Most general second-order equation for mechanical system of this type (according to Newton's law) is:

$$\frac{d^2y}{dt^2} = \frac{1}{m}F(y, \dot{y}, t) \tag{1}$$

- If we can express forcing function $F(y, \dot{y}, t)$ as sum of two functions $F_1(y, \dot{y})$ and $F_2(t)$, then we can rearrange into input-output form:

$$\frac{d^2y}{dt^2} - \frac{1}{m}F_1(y, \dot{y}) = \frac{1}{m}F_2(t)$$

Dynamic Systems

Dynamic Systems

- In engineering applications, we are often interested not only in modeling system dynamics, but also in measuring output or resulting behavior of system in response to inputs, or dependency on initial conditions

Dynamic Systems

- In engineering applications, we are often interested not only in modeling system dynamics, but also in measuring output or resulting behavior of system in response to inputs, or dependency on initial conditions
- But outputs are not always directly observable!!

Dynamic Systems

- In engineering applications, we are often interested not only in modeling system dynamics, but also in measuring output or resulting behavior of system in response to inputs, or dependency on initial conditions
- But outputs are not always directly observable!!
- Thus, in specifying dynamic systems, we often add second equation, usually algebraic or functional in nature, that relates output y (and possibly the input u) to some measured or observed quantities

$$z = h(y) + g(u)$$

Dynamic Systems

- In engineering applications, we are often interested not only in modeling system dynamics, but also in measuring output or resulting behavior of system in response to inputs, or dependency on initial conditions
- But outputs are not always directly observable!!
- Thus, in specifying dynamic systems, we often add second equation, usually algebraic or functional in nature, that relates output y (and possibly the input u) to some measured or observed quantities

$$z = h(y) + g(u)$$

The Concept of State

The Concept of State

- To determine behavior of deterministic system at any future time requires knowledge of two things:
  1. Equations governing system dynamics
  2. Initial conditions

The Concept of State

- To determine behavior of deterministic system at any future time requires knowledge of two things:
  1. Equations governing system dynamics
  2. Initial conditions
- But initial conditions for n^{th}-order continuous dynamic system are simply values of first $n-1$ derivatives at initial time

The Concept of State

- To determine behavior of deterministic system at any future time requires knowledge of two things:
  1. Equations governing system dynamics
  2. Initial conditions
- But initial conditions for n^{th}-order continuous dynamic system are simply values of first $n-1$ derivatives at initial time
- "How is a dynamic system (linear or nonlinear) described? The fundamental concept is the notion of the state. By this is meant, intuitively, some quantitative information (a set of numbers, a function, etc.) which is the least amount of data one has to know about the past behavior of the system in order to predict its future behavior." Rudolf Kalman, in seminal paper (1960)

The Concept of State

- To determine behavior of deterministic system at any future time requires knowledge of two things:
  1. Equations governing system dynamics
  2. Initial conditions
- But initial conditions for n^{th}-order continuous dynamic system are simply values of first $n-1$ derivatives at initial time
- "How is a dynamic system (linear or nonlinear) described? The fundamental concept is the notion of the state. By this is meant, intuitively, some quantitative information (a set of numbers, a function, etc.) which is the least amount of data one has to know about the past behavior of the system in order to predict its future behavior." Rudolf Kalman, in seminal paper (1960)

The Concept of State

The Concept of State

- At any point in time, we can think of instantaneous values of first $n-1$ derivatives as initial conditions for all future dynamic evolution

The Concept of State

- At any point in time, we can think of instantaneous values of first $n-1$ derivatives as initial conditions for all future dynamic evolution
- These values capture the "state" of the dynamic system at that time

The Concept of State

- At any point in time, we can think of instantaneous values of first $n-1$ derivatives as initial conditions for all future dynamic evolution
- These values capture the "state" of the dynamic system at that time

The Concept of State

The Concept of State

- This concept of "state" is reinforced by considering the Taylor series expansion of a function about some point in time.

The Concept of State

- This concept of "state" is reinforced by considering the Taylor series expansion of a function about some point in time.
- Taylor's theorem says that if a function $y(t)$ is sufficiently well-behaved in a neighborhood about a point t_0, then the value of y at any other point t in that neighborhood can be determined from the values of all of its derivatives at the given point t_0.

The Concept of State

- This concept of "state" is reinforced by considering the Taylor series expansion of a function about some point in time.
- Taylor's theorem says that if a function $y(t)$ is sufficiently well-behaved in a neighborhood about a point t_0, then the value of y at any other point t in that neighborhood can be determined from the values of all of its derivatives at the given point t_0.

$$y(t) = y(t_0) + \dot{y}(t_0)(t - t_0) + \frac{1}{2!}\ddot{y}(t_0)(t - t_0)^2 + \cdots$$

The Concept of State

- This concept of "state" is reinforced by considering the Taylor series expansion of a function about some point in time.
- Taylor's theorem says that if a function $y(t)$ is sufficiently well-behaved in a neighborhood about a point t_0, then the value of y at any other point t in that neighborhood can be determined from the values of all of its derivatives at the given point t_0.

$$y(t) = y(t_0) + \dot{y}(t_0)(t - t_0) + \frac{1}{2!}\ddot{y}(t_0)(t - t_0)^2 + \cdots$$

- Note that this works for time points in the past as well as those in the future, provided they are sufficiently close to t_0.

The Concept of State

- This concept of "state" is reinforced by considering the Taylor series expansion of a function about some point in time.
- Taylor's theorem says that if a function $y(t)$ is sufficiently well-behaved in a neighborhood about a point t_0, then the value of y at any other point t in that neighborhood can be determined from the values of all of its derivatives at the given point t_0.

$$y(t) = y(t_0) + \dot{y}(t_0)(t - t_0) + \frac{1}{2!}\ddot{y}(t_0)(t - t_0)^2 + \cdots$$

- Note that this works for time points in the past as well as those in the future, provided they are sufficiently close to t_0.

The Concept of State

The Concept of State

- In the language of dynamic systems, the quantity $t - t_0$ (if > 0) is simply the time elapsed since the initial time t_0 (otherwise, it is the negative duration from t_0 to a past time point t).

The Concept of State

- In the language of dynamic systems, the quantity $t - t_0$ (if > 0) is simply the time elapsed since the initial time t_0 (otherwise, it is the negative duration from t_0 to a past time point t).
- Thus, if you have knowledge of the state at time t_0, i.e., if you know the values of all the (infinitely many) derivatives, then you can determine the value of the function at any later time (sufficiently close to t_0).

The Concept of State

- In the language of dynamic systems, the quantity $t - t_0$ (if > 0) is simply the time elapsed since the initial time t_0 (otherwise, it is the negative duration from t_0 to a past time point t).
- Thus, if you have knowledge of the state at time t_0, i.e., if you know the values of all the (infinitely many) derivatives, then you can determine the value of the function at any later time (sufficiently close to t_0).
- This is because all of the dynamic information is contained in the derivatives.

The Concept of State

- In the language of dynamic systems, the quantity $t - t_0$ (if > 0) is simply the time elapsed since the initial time t_0 (otherwise, it is the negative duration from t_0 to a past time point t).
- Thus, if you have knowledge of the state at time t_0, i.e., if you know the values of all the (infinitely many) derivatives, then you can determine the value of the function at any later time (sufficiently close to t_0).
- This is because all of the dynamic information is contained in the derivatives.
- Of course, if $y(t)$ is infinitely differentiable, then you can also express each of the derivatives of y at time t in terms of their values at time t_0 by means of their Taylor series, and thus from the state at t_0 you can compute the state at time t.

The Concept of State

- In the language of dynamic systems, the quantity $t - t_0$ (if > 0) is simply the time elapsed since the initial time t_0 (otherwise, it is the negative duration from t_0 to a past time point t).
- Thus, if you have knowledge of the state at time t_0, i.e., if you know the values of all the (infinitely many) derivatives, then you can determine the value of the function at any later time (sufficiently close to t_0).
- This is because all of the dynamic information is contained in the derivatives.
- Of course, if $y(t)$ is infinitely differentiable, then you can also express each of the derivatives of y at time t in terms of their values at time t_0 by means of their Taylor series, and thus from the state at t_0 you can compute the state at time t.

Concept of State

Concept of State

$$\begin{bmatrix} y(t) \\ D_t^1 y(t) \\ \vdots \\ D_t^n y(t) \end{bmatrix} \approx \begin{bmatrix} \frac{1}{0!} & \frac{n}{1!} & \cdots & \cdots & \frac{n^n}{(n)!} \\ 0 & \frac{1}{0!} & \frac{n}{1!} & \cdots & \frac{n^{(n-1)}}{(n-1)!} \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ 0 & 0 & \cdots & \cdots & 1 \end{bmatrix} \begin{bmatrix} n(t_0) \\ D_t[n](t_0) \\ \vdots \\ D_t^n y(t_0) \end{bmatrix} (t - t_0) \quad (2)$$

Concept of State

$$\begin{bmatrix} y(t) \\ D_t^1 y(t) \\ \vdots \\ D_t^n y(t) \end{bmatrix} \approx$$

$$\begin{bmatrix} \frac{1}{0!} & \frac{n}{1!} & \cdots & \cdots & \frac{n^n}{(n)!} \\ 0 & \frac{1}{0!} & \frac{n}{1!} & \cdots & \frac{n^{(n-1)}}{(n-1)!} \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ 0 & 0 & \cdots & \cdots & 1 \end{bmatrix} \begin{bmatrix} n(t_0) \\ D_t[n](t_0) \\ \vdots \\ D_t^n y(t_0) \end{bmatrix} (t - t_0) \quad (2)$$

The Concept of State

The Concept of State

- Concept of state involves in some sense the instantaneous internal arrangement of a system

The Concept of State

- Concept of state involves in some sense the instantaneous internal arrangement of a system
- In practical situations, the state variables may represent attributes of the system that we wish to know something about

The Concept of State

- Concept of state involves in some sense the instantaneous internal arrangement of a system
- In practical situations, the state variables may represent attributes of the system that we wish to know something about
- Also intuitively, state variables are usually naturally coupled in some sense; this is obvious when they are derived from a single differential equation in which all quantities are derivatives of a single variable (the output variable)

The Concept of State

- Concept of state involves in some sense the instantaneous internal arrangement of a system
- In practical situations, the state variables may represent attributes of the system that we wish to know something about
- Also intuitively, state variables are usually naturally coupled in some sense; this is obvious when they are derived from a single differential equation in which all quantities are derivatives of a single variable (the output variable)

The Concept of State

The Concept of State

- Mechanical systems like a moving particle have three simultaneous attributes: position, velocity, and acceleration.

The Concept of State

- Mechanical systems like a moving particle have three simultaneous attributes: position, velocity, and acceleration.
- Newton tells us that acceleration is often a function of position, velocity and perhaps time.

The Concept of State

- Mechanical systems like a moving particle have three simultaneous attributes: position, velocity, and acceleration.
- Newton tells us that acceleration is often a function of position, velocity and perhaps time.
- However, as we just saw, a force that acts on a system as a function of time only should be considered an external forcing function, i.e., an input to the system.

The Concept of State

- Mechanical systems like a moving particle have three simultaneous attributes: position, velocity, and acceleration.
- Newton tells us that acceleration is often a function of position, velocity and perhaps time.
- However, as we just saw, a force that acts on a system as a function of time only should be considered an external forcing function, i.e., an input to the system.
- Thus we are led to think of position and velocity as the primary state variables of interest.

State Space Representation

State Space Representation

- A general n-th order linear differential equation can be expressed as a system of first-order differential equations.

State Space Representation

- A general n-th order linear differential equation can be expressed as a system of first-order differential equations.
- Ignoring the input u for the moment, consider:

$$p(D)y = D^n y + a_{n-1} D^{n-1} y + \cdots + a_1 Dy + a_0 y,$$

where we have put the polynomial in normal form, i.e., such that its leading coefficient is 1.

State Space Representation

- A general n-th order linear differential equation can be expressed as a system of first-order differential equations.
- Ignoring the input u for the moment, consider:

$$p(D)y = D^n y + a_{n-1} D^{n-1} y + \cdots + a_1 Dy + a_0 y,$$

where we have put the polynomial in normal form, i.e., such that its leading coefficient is 1.

State Space Representation

State Space Representation

- If we consider y and its first $n-1$ derivatives as the states of this system, then we have n first order equations. Setting $x_1 = y, x_2 = Dy, \ldots$, we can write

$$
\begin{aligned}
x_1(t) &= y(t) \\
x_2(t) &= Dy(t) \\
&\ldots \\
x_n(t) &= D^{n-1}y(t).
\end{aligned}
$$

State Space Representation

- If we consider y and its first $n-1$ derivatives as the states of this system, then we have n first order equations. Setting $x_1 = y, x_2 = Dy, \ldots$, we can write

$$\begin{aligned} x_1(t) &= y(t) \\ x_2(t) &= Dy(t) \\ &\cdots \\ x_n(t) &= D^{n-1}y(t). \end{aligned}$$

- The functions $x_i(t)$ are called the state variables.

State Space Representation

- If we consider y and its first $n-1$ derivatives as the states of this system, then we have n first order equations. Setting $x_1 = y, x_2 = Dy, \ldots$, we can write

$$\begin{aligned} x_1(t) &= y(t) \\ x_2(t) &= Dy(t) \\ &\cdots \\ x_n(t) &= D^{n-1}y(t). \end{aligned}$$

- The functions $x_i(t)$ are called the state variables.

State Space Representation

State Space Representation

- Taking the derivative of these n state variables and making use of the original differential equation yields:

$$
\begin{aligned}
Dx_1(t) &= x_2(t) \\
Dx_2(t) &= x_3(t) \\
&\dots \\
Dx_n(t) &= -a_0 x_1(t) - a_1 x_2(t) - \dots - a_{n-1} x_n(t).
\end{aligned}
$$

State Space Representation

- Taking the derivative of these n state variables and making use of the original differential equation yields:

$$\begin{aligned} Dx_1(t) &= x_2(t) \\ Dx_2(t) &= x_3(t) \\ &\cdots \\ Dx_n(t) &= -a_0 x_1(t) - a_1 x_2(t) - \cdots - a_{n-1} x_n(t). \end{aligned}$$

- These n equations can be written more compactly in vector matrix form: $Dx(t) = Ax(t)$ or $\dot{x}(t) = Ax(t)$.

State Space Representation

- Taking the derivative of these n state variables and making use of the original differential equation yields:

$$\begin{aligned} Dx_1(t) &= x_2(t) \\ Dx_2(t) &= x_3(t) \\ &\cdots \\ Dx_n(t) &= -a_0 x_1(t) - a_1 x_2(t) - \cdots - a_{n-1} x_n(t). \end{aligned}$$

- These n equations can be written more compactly in vector matrix form: $Dx(t) = Ax(t)$ or $\dot{x}(t) = Ax(t)$.

State Space Representation

State Space Representation

- In the equation $\dot{x}(t) = Ax(t)$, we have:

$$x(t) = \begin{bmatrix} x_1(t) \\ x_2(t) \\ \vdots \\ x_n(t) \end{bmatrix}$$

and

$$A = \begin{bmatrix} 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \\ \vdots & & \ddots & & \vdots \\ 0 & 0 & 0 & 0 & 1 \\ -a_0 & -a_1 & \cdots & -a_{n-2} & -a_{n-1} \end{bmatrix},$$

the companion matrix of the differential equation.

State Space Representation

- In the equation $\dot{x}(t) = Ax(t)$, we have:

$$x(t) = \begin{bmatrix} x_1(t) \\ x_2(t) \\ \vdots \\ x_n(t) \end{bmatrix}$$

and

$$A = \begin{bmatrix} 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \\ \vdots & & \ddots & & \vdots \\ 0 & 0 & 0 & 0 & 1 \\ -a_0 & -a_1 & \cdots & -a_{n-2} & -a_{n-1} \end{bmatrix},$$

the companion matrix of the differential equation.

State Space Representation

State Space Representation

- If there is an input function u and it is present via its derivatives, then we can write:

$$\dot{x} = Ax + Bu \tag{3}$$

forming B in much the same way we formed A, and suppressing the time variable t.

State Space Representation

- If there is an input function u and it is present via its derivatives, then we can write:

$$\dot{x} = Ax + Bu \tag{3}$$

  forming B in much the same way we formed A, and suppressing the time variable t.
- Equation (3) expresses the system's state dynamics.
- Later we will include white noise as an input. However, that will make the differential equations stochastic, and we are not yet ready for that.

State Space Representation

- If there is an input function u and it is present via its derivatives, then we can write:

$$\dot{x} = Ax + Bu \tag{3}$$

  forming B in much the same way we formed A, and suppressing the time variable t.
- Equation (3) expresses the system's state dynamics.
- Later we will include white noise as an input. However, that will make the differential equations stochastic, and we are not yet ready for that.

State Space Representation

State Space Representation

- To obtain the original output y as a function of the internal state of the system, we require one additional vector-matrix equation:

$$y = Cx + Du \tag{4}$$

where the matrix D represents a **direct feedthrough** of the input to the output.

State Space Representation

- To obtain the original output y as a function of the internal state of the system, we require one additional vector-matrix equation:

$$y = Cx + Du \tag{4}$$

  where the matrix D represents a **direct feedthrough** of the input to the output.
- The output equation is derived directly from the b_i coefficients in the original matrix differential equation.

State Space Representation

- To obtain the original output y as a function of the internal state of the system, we require one additional vector-matrix equation:

$$y = Cx + Du \tag{4}$$

  where the matrix D represents a **direct feedthrough** of the input to the output.
- The output equation is derived directly from the b_i coefficients in the original matrix differential equation.

State Space Representation

State Space Representation

- Kalman filter theory generally assumes $D = 0$ in the output equation and uses a measurement model instead:

$$z = Hx \tag{5}$$

State Space Representation

- Kalman filter theory generally assumes $D = 0$ in the output equation and uses a measurement model instead:

$$z = Hx \tag{5}$$

- Note that H implicitly includes C as a factor.

State Space Representation

- Kalman filter theory generally assumes $D = 0$ in the output equation and uses a measurement model instead:

$$z = Hx \tag{5}$$

- Note that H implicitly includes C as a factor.
- Note that we have not placed any restrictions on the size of the measurement column vector z. As long as H has n columns it can have as many rows as desired.

State Space Representation

- Kalman filter theory generally assumes $D = 0$ in the output equation and uses a measurement model instead:

$$z = Hx \tag{5}$$

- Note that H implicitly includes C as a factor.
- Note that we have not placed any restrictions on the size of the measurement column vector z. As long as H has n columns it can have as many rows as desired.

State Space Representation: Example

State Space Representation: Example

- Let's consider once again the simple harmonic oscillator:

$$m\frac{d^2y}{dt^2} + ky = 0$$

and convert this to state space form:

$$x_1 = y$$
$$x_2 = Dy$$

and $\dot{x}_2 = -\frac{k}{m}x_1 + 0 \cdot x_2$.

State Space Representation: Example

- Let's consider once again the simple harmonic oscillator:

$$m\frac{d^2y}{dt^2} + ky = 0$$

and convert this to state space form:

$$x_1 = y$$
$$x_2 = Dy$$

and $\dot{x}_2 = -\frac{k}{m}x_1 + 0 \cdot x_2$.

- Therefore, in matrix and vector form:

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -k/m & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}.$$

State Space Representation: Example

- Let's consider once again the simple harmonic oscillator:

$$m\frac{d^2y}{dt^2} + ky = 0$$

and convert this to state space form:

$$x_1 = y$$
$$x_2 = Dy$$

and $\dot{x}_2 = -\frac{k}{m}x_1 + 0 \cdot x_2$.

- Therefore, in matrix and vector form:

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -k/m & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}.$$

State Space Representation: Example

State Space Representation: Example

- For a measurement model, we could consider a camera with a strobe flash that periodically records the instantaneous position of the sliding mass.

State Space Representation: Example

- For a measurement model, we could consider a camera with a strobe flash that periodically records the instantaneous position of the sliding mass.
- In this case, only the position would be observable and we would write the measurement or observation model as:

$$z = \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}.$$

(Remember that we are ignoring both process and measurement noise.)

State Space Representation: Example

- For a measurement model, we could consider a camera with a strobe flash that periodically records the instantaneous position of the sliding mass.
- In this case, only the position would be observable and we would write the measurement or observation model as:

$$z = \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}.$$

(Remember that we are ignoring both process and measurement noise.)

State Space Representation

State Space Representation

- A single n^{th} order ordinary differential equation presents only one way of deriving a state space representation for a dynamic system

State Space Representation

- A single n^{th} order ordinary differential equation presents only one way of deriving a state space representation for a dynamic system
- Can also derive state space models directly from systems of coupled first order ODEs

State Space Representation

- A single n^{th} order ordinary differential equation presents only one way of deriving a state space representation for a dynamic system
- Can also derive state space models directly from systems of coupled first order ODEs
  - Real world systems consisting of electrical, mechanical, thermal, hydraulic components are coupled in this way: e.g., a DC motor driving an inertial load

State Space Representation

- A single n^{th} order ordinary differential equation presents only one way of deriving a state space representation for a dynamic system
- Can also derive state space models directly from systems of coupled first order ODEs
  - Real world systems consisting of electrical, mechanical, thermal, hydraulic components are coupled in this way: e.g., a DC motor driving an inertial load
- In discrete time, ordinary difference equations also lead to state space models

State Space Representation

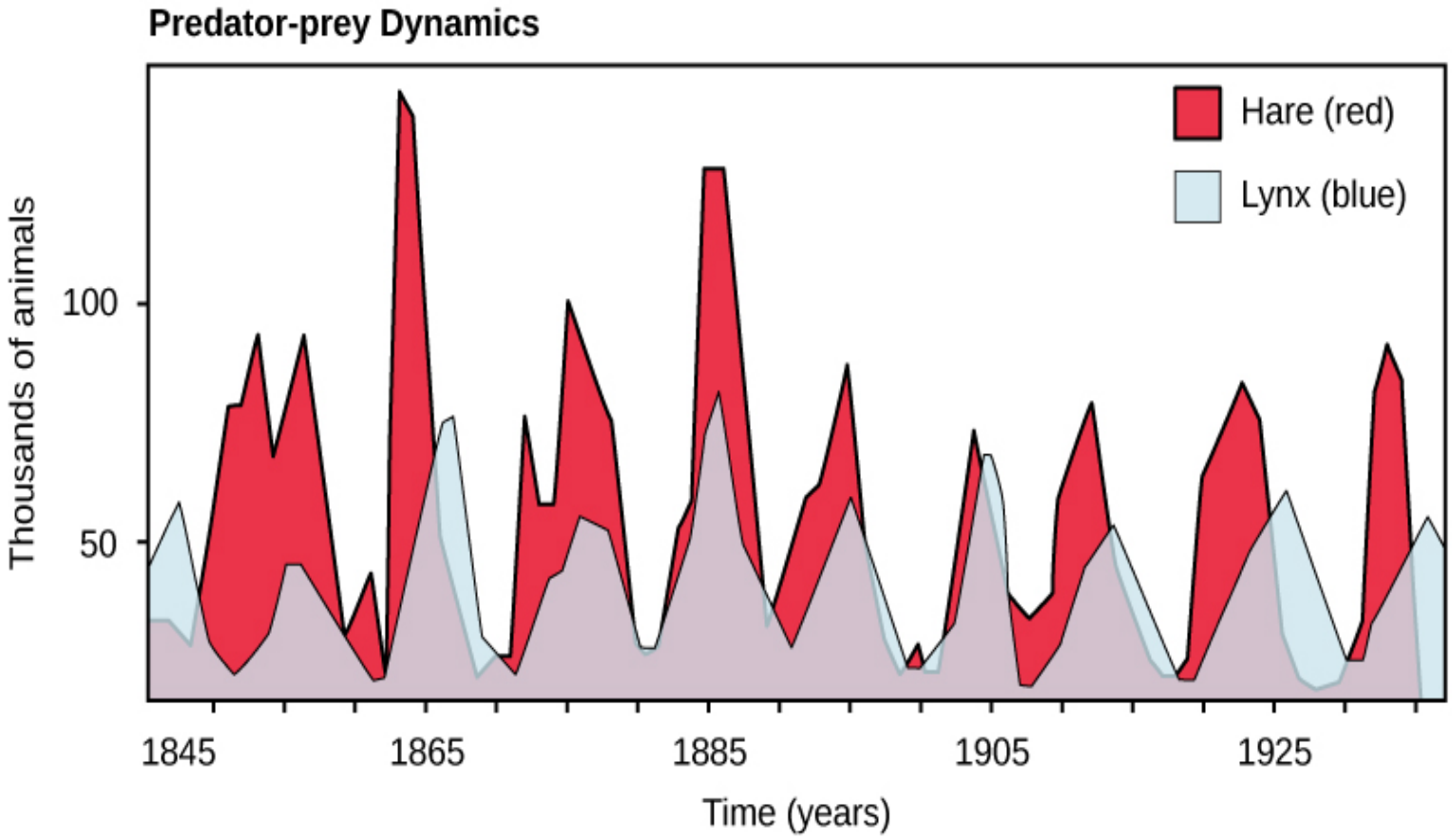
- A single n^{th} order ordinary differential equation presents only one way of deriving a state space representation for a dynamic system
- Can also derive state space models directly from systems of coupled first order ODEs
  - Real world systems consisting of electrical, mechanical, thermal, hydraulic components are coupled in this way: e.g., a DC motor driving an inertial load
- In discrete time, ordinary difference equations also lead to state space models

State Space Representation

Example: Predator-Prey Equations, Real World

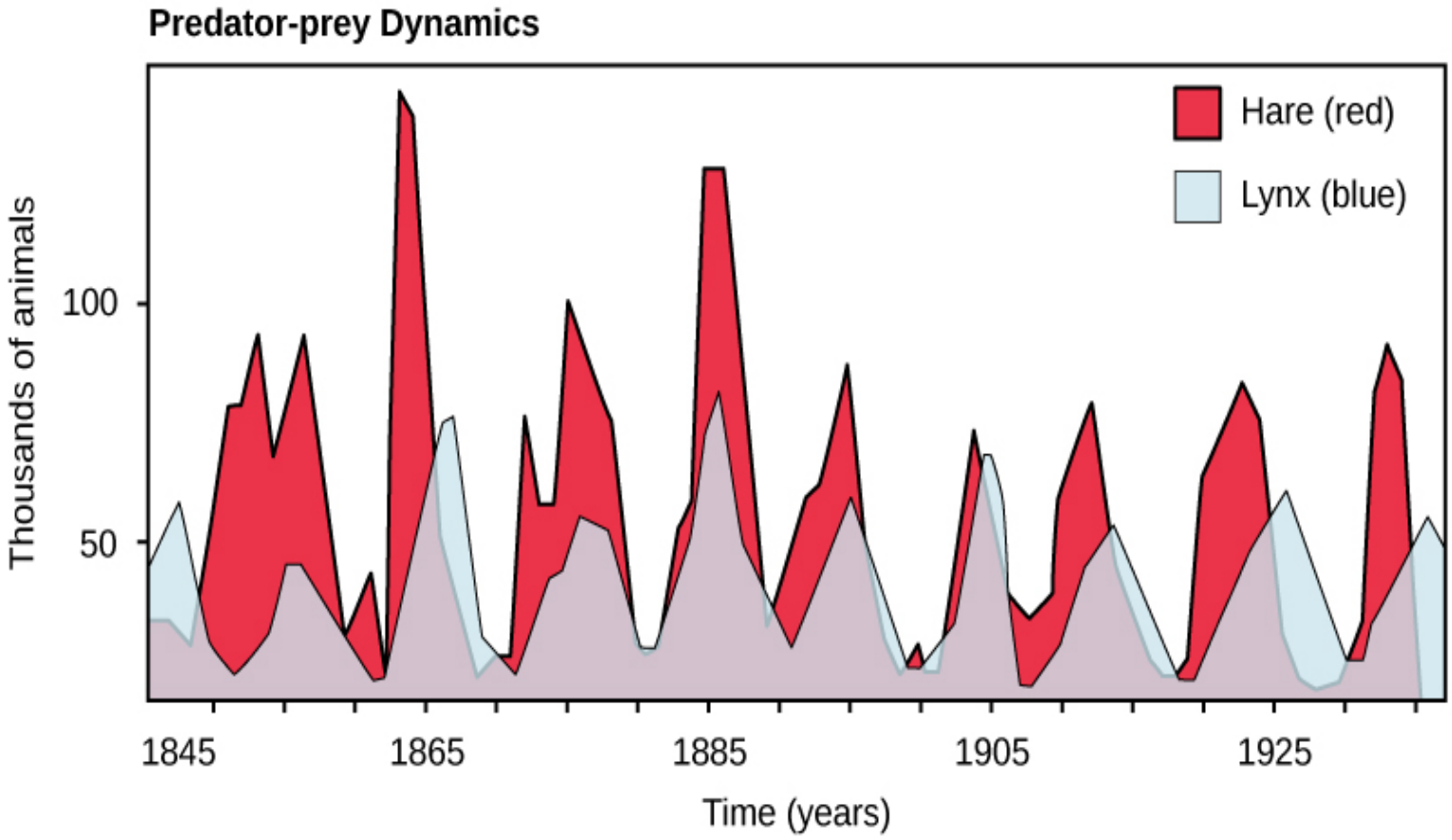
State Space Representation

Example: Predator-Prey Equations, Real World



State Space Representation

Example: Predator-Prey Equations, Real World



State Space Representation

Example: Predator-Prey Equations

State Space Representation

Example: Predator-Prey Equations

- Sufficient food is always available for the prey population

State Space Representation

Example: Predator-Prey Equations

- Sufficient food is always available for the prey population
- Food supply of predator population depends only on size of prey population (predators eat only one kind of thing)

State Space Representation

Example: Predator-Prey Equations

- Sufficient food is always available for the prey population
- Food supply of predator population depends only on size of prey population (predators eat only one kind of thing)
- Current rate of change of population is proportional to its current size

State Space Representation

Example: Predator-Prey Equations

- Sufficient food is always available for the prey population
- Food supply of predator population depends only on size of prey population (predators eat only one kind of thing)
- Current rate of change of population is proportional to its current size
- Environment does not change to favor one species over another

State Space Representation

Example: Predator-Prey Equations

- Sufficient food is always available for the prey population
- Food supply of predator population depends only on size of prey population (predators eat only one kind of thing)
- Current rate of change of population is proportional to its current size
- Environment does not change to favor one species over another
- Genetic adaptation is negligible

State Space Representation

Example: Predator-Prey Equations

- Sufficient food is always available for the prey population
- Food supply of predator population depends only on size of prey population (predators eat only one kind of thing)
- Current rate of change of population is proportional to its current size
- Environment does not change to favor one species over another
- Genetic adaptation is negligible
- Predators are insatiable

State Space Representation

Example: Predator-Prey Equations

- Sufficient food is always available for the prey population
- Food supply of predator population depends only on size of prey population (predators eat only one kind of thing)
- Current rate of change of population is proportional to its current size
- Environment does not change to favor one species over another
- Genetic adaptation is negligible
- Predators are insatiable

State Space Representation

Example: Predator-Prey Equations

State Space Representation

Example: Predator-Prey Equations

- Lotka-Volterra Equations

State Space Representation

Example: Predator-Prey Equations

- Lotka-Volterra Equations
- Predator-Prey Population Dynamics

  Prey: $\dot{x}_1(t) = ax_1(t) - bx_2(t)$

  Predators: $\dot{x}_2(t) = cx_1(t) - dx_2(t)$

State Space Representation

Example: Predator-Prey Equations

- Lotka-Volterra Equations
- Predator-Prey Population Dynamics
  Prey: $\dot{x}_1(t) = ax_1(t) - bx_2(t)$
  Predators: $\dot{x}_2(t) = cx_1(t) - dx_2(t)$
- i.e., in state space form:

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \end{bmatrix} = \begin{bmatrix} a & -b \\ c & -d \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

State Space Representation

Example: Predator-Prey Equations

- Lotka-Volterra Equations
- Predator-Prey Population Dynamics
  Prey: $\dot{x}_1(t) = ax_1(t) - bx_2(t)$
  Predators: $\dot{x}_2(t) = cx_1(t) - dx_2(t)$
- i.e., in state space form:

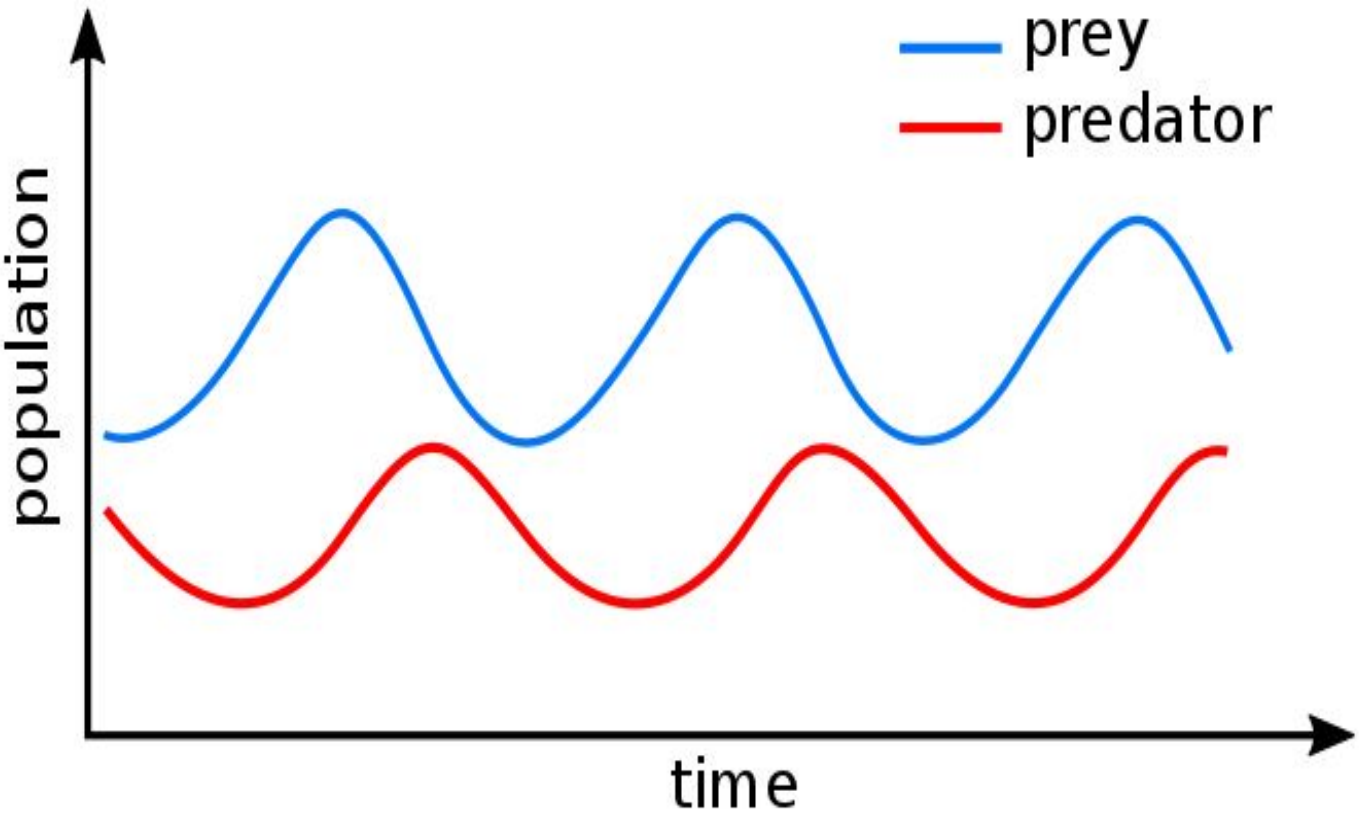
$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \end{bmatrix} = \begin{bmatrix} a & -b \\ c & -d \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

State Space Representation

Example: Predator-Prey Equations, Simulated

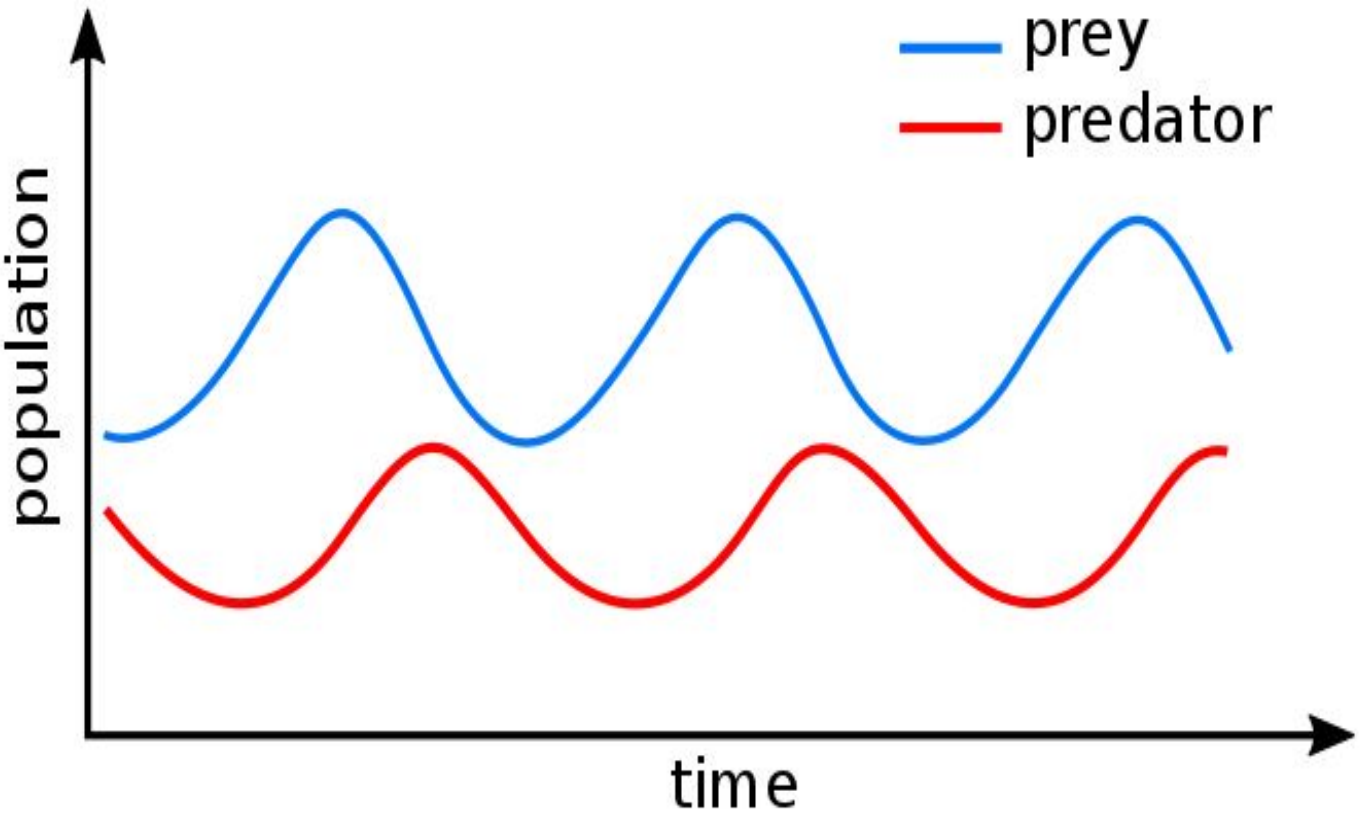
State Space Representation

Example: Predator-Prey Equations, Simulated



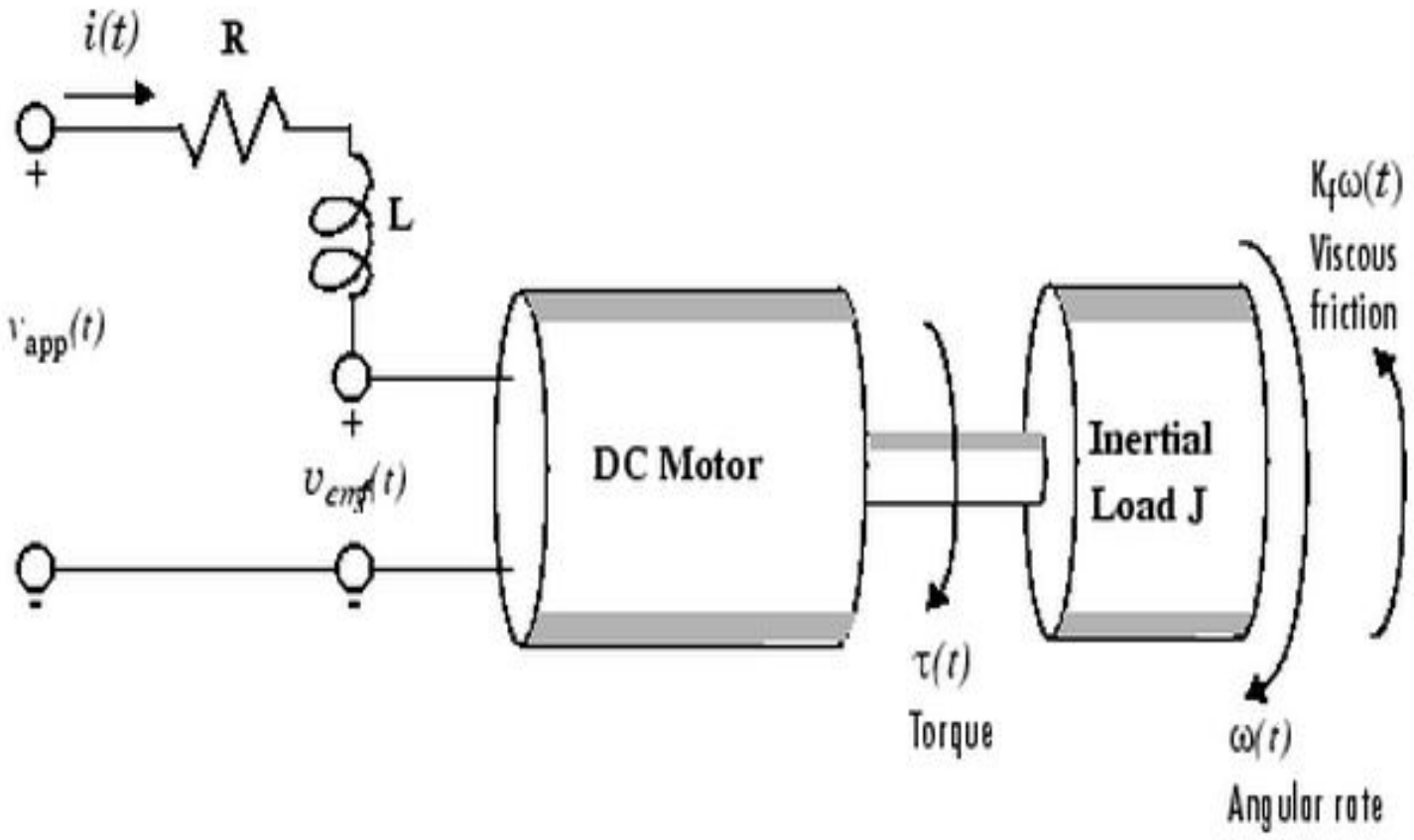
State Space Representation

Example: Predator-Prey Equations, Simulated



State Space Representation

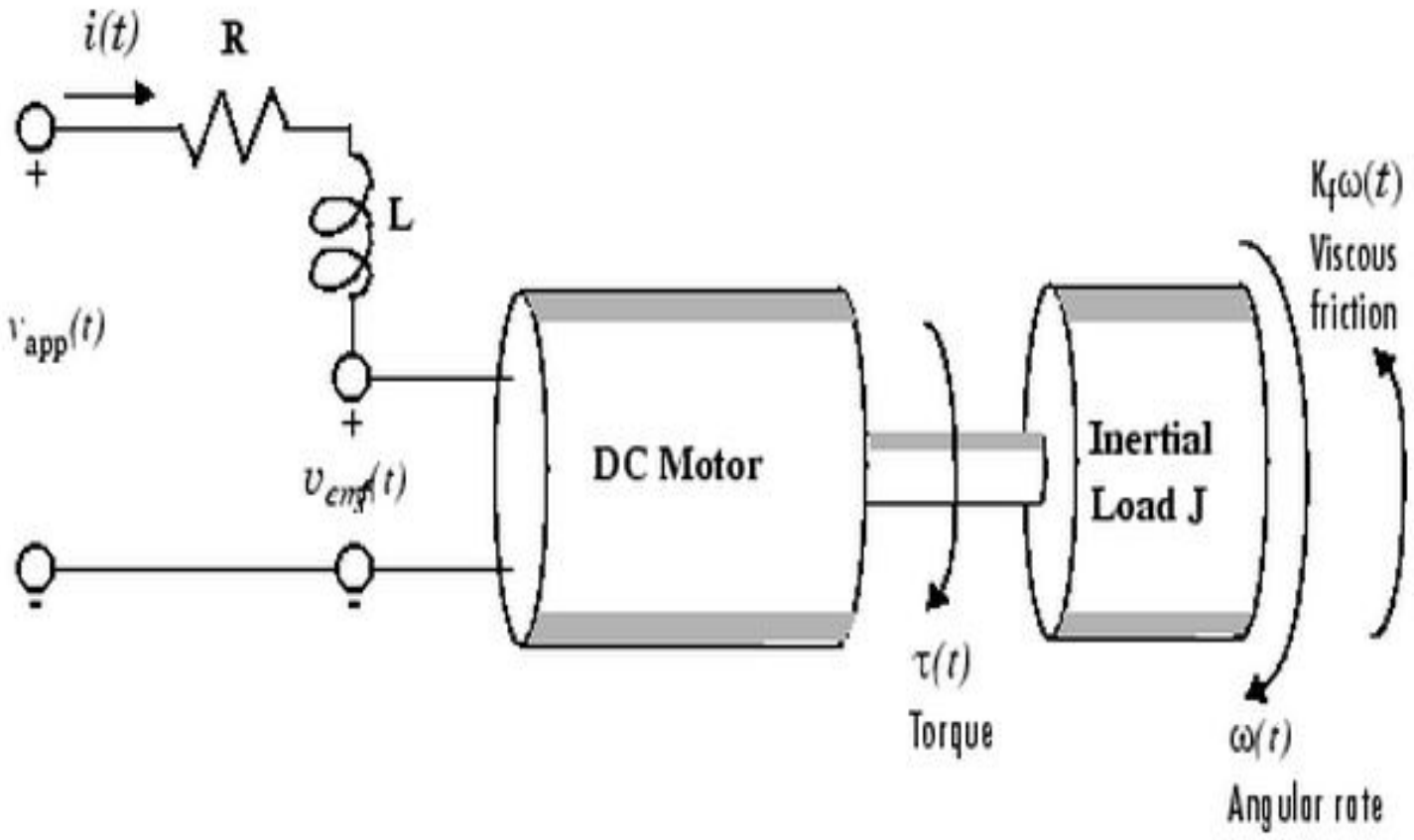
Example: DC Motor with Inertial Load



Source: MATLAB Tutorial

State Space Representation

Example: DC Motor with Inertial Load



Source: MATLAB Tutorial

State Space Representation

Examples: DC Motor with Inertial Load

State Space Representation

Examples: DC Motor with Inertial Load

- Electromechanical: Torque τ proportional to current i induced by applied voltage: $\tau(t) = K_m i(t)$,
  whereK_m is armature constant (related to magnetic field strength, number of turns of wire, etc.)

State Space Representation

Examples: DC Motor with Inertial Load

- Electromechanical: Torque τ proportional to current i induced by applied voltage: $\tau(t) = K_m i(t)$,
  whereK_m is armature constant (related to magnetic field strength, number of turns of wire, etc.)
- Electomechanical Back EMF: $v_{emf}(t) = K_b \omega(t)$,
  where K_b is EMF constant

State Space Representation

Examples: DC Motor with Inertial Load

- Electromechanical: Torque τ proportional to current i induced by applied voltage: $\tau(t) = K_m i(t)$,
  whereK_m is armature constant (related to magnetic field strength, number of turns of wire, etc.)
- Electomechanical Back EMF: $v_{emf}(t) = K_b \omega(t)$,
  where K_b is EMF constant
- Mechanical (from Newton's laws): $J\frac{d\omega}{dt} = -K_f\omega(t) + K_m i(t)$,
  where $K_f\omega$ is linear approximation of viscouous friction

State Space Representation

Examples: DC Motor with Inertial Load

- Electromechanical: Torque τ proportional to current i induced by applied voltage: $\tau(t) = K_m i(t)$,
  whereK_m is armature constant (related to magnetic field strength, number of turns of wire, etc.)
- Electomechanical Back EMF: $v_{emf}(t) = K_b \omega(t)$,
  where K_b is EMF constant
- Mechanical (from Newton's laws): $J\frac{d\omega}{dt} = -K_f\omega(t) + K_m i(t)$,
  where $K_f\omega$ is linear approximation of viscouous friction
- Electrical:
  $v_{app}(t) = L\frac{di}{dt} + Ri(t) + v_{emf}(t) = L\frac{di}{dt} + Ri(t) + K_b\omega(t)$,
  where v_{app} is applied voltage, L is inductance, and R is resistance

State Space Representation

Examples: DC Motor with Inertial Load

- Electromechanical: Torque τ proportional to current i induced by applied voltage: $\tau(t) = K_m i(t)$,
  whereK_m is armature constant (related to magnetic field strength, number of turns of wire, etc.)
- Electomechanical Back EMF: $v_{emf}(t) = K_b \omega(t)$,
  where K_b is EMF constant
- Mechanical (from Newton's laws): $J\frac{d\omega}{dt} = -K_f\omega(t) + K_m i(t)$,
  where $K_f\omega$ is linear approximation of viscouous friction
- Electrical:
  $v_{app}(t) = L\frac{di}{dt} + Ri(t) + v_{emf}(t) = L\frac{di}{dt} + Ri(t) + K_b\omega(t)$,
  where v_{app} is applied voltage, L is inductance, and R is resistance

State Space Representation

Examples: DC Motor with Inertial Load

State Space Representation

Examples: DC Motor with Inertial Load

- Equations for $\frac{di}{dt}$ and $\frac{d\omega}{dt}$ lead to state space form

State Space Representation

Examples: DC Motor with Inertial Load

- Equations for $\frac{di}{dt}$ and $\frac{d\omega}{dt}$ lead to state space form
- $$\frac{d}{dt}\begin{bmatrix} i \\ \omega \end{bmatrix} = \begin{bmatrix} -\frac{R}{L} & -\frac{K_b}{L} \\ \frac{K_m}{J} & -\frac{K_f}{J} \end{bmatrix} \begin{bmatrix} i \\ \omega \end{bmatrix} + \begin{bmatrix} 1/L \\ 0 \end{bmatrix} v_{app}(t)$$

State Space Representation

Examples: DC Motor with Inertial Load

- Equations for $\frac{di}{dt}$ and $\frac{d\omega}{dt}$ lead to state space form
- $$\frac{d}{dt}\begin{bmatrix} i \\ \omega \end{bmatrix} = \begin{bmatrix} -\frac{R}{L} & -\frac{K_b}{L} \\ \frac{K_m}{J} & -\frac{K_f}{J} \end{bmatrix} \begin{bmatrix} i \\ \omega \end{bmatrix} + \begin{bmatrix} 1/L \\ 0 \end{bmatrix} v_{app}(t)$$
- Note the presence of control input $v_{app}(t)$

State Space Representation

Examples: DC Motor with Inertial Load

- Equations for $\frac{di}{dt}$ and $\frac{d\omega}{dt}$ lead to state space form
- $$\frac{d}{dt}\begin{bmatrix} i \\ \omega \end{bmatrix} = \begin{bmatrix} -\frac{R}{L} & -\frac{K_b}{L} \\ \frac{K_m}{J} & -\frac{K_f}{J} \end{bmatrix} \begin{bmatrix} i \\ \omega \end{bmatrix} + \begin{bmatrix} 1/L \\ 0 \end{bmatrix} v_{app}(t)$$
- Note the presence of control input $v_{app}(t)$

Ordinary Difference Equations

Ordinary Difference Equations

- Auto-Regressive Moving Average (ARMA)

Ordinary Difference Equations

- Auto-Regressive Moving Average (ARMA)
- n^{th} derivative corresponds to n-step advance:

Ordinary Difference Equations

- Auto-Regressive Moving Average (ARMA)
- n^{th} derivative corresponds to n-step advance:

$$\sum_{i=0}^{n} a_{n-i} y(k+n-i) = \sum_{i=0}^{m} b_{m-i} u(k+m-i) \qquad (6)$$

Ordinary Difference Equations

- Auto-Regressive Moving Average (ARMA)
- n^{th} derivative corresponds to n-step advance:

$$\sum_{i=0}^{n} a_{n-i} y(k+n-i) = \sum_{i=0}^{m} b_{m-i} u(k+m-i) \qquad (6)$$

- The left side of Eq. (6) is the **Auto-Regressive** part

Ordinary Difference Equations

- Auto-Regressive Moving Average (ARMA)
- n^{th} derivative corresponds to n-step advance:

$$\sum_{i=0}^{n} a_{n-i} y(k+n-i) = \sum_{i=0}^{m} b_{m-i} u(k+m-i) \qquad (6)$$

- The left side of Eq. (6) is the **Auto-Regressive** part
- The right side of Eq. (6) is the **Moving Average** part

Ordinary Difference Equations

- Auto-Regressive Moving Average (ARMA)
- n^{th} derivative corresponds to n-step advance:

$$\sum_{i=0}^{n} a_{n-i} y(k+n-i) = \sum_{i=0}^{m} b_{m-i} u(k+m-i) \qquad (6)$$

- The left side of Eq. (6) is the **Auto-Regressive** part
- The right side of Eq. (6) is the **Moving Average** part

Ordinary Difference Equations

Example

Ordinary Difference Equations

Example

- Let $n = 3$, $m = 2$ and consider the difference equation:

Ordinary Difference Equations

Example

- Let $n = 3$, $m = 2$ and consider the difference equation:

$$y(k+3) + 1.5y(k+2) + 7y(k+1) - 3.2y(k) = 23u(k+2) - 5u(k+1) + 0.5u(k) \quad (7)$$

Ordinary Difference Equations

Example

- Let $n = 3$, $m = 2$ and consider the difference equation:

$$y(k+3) + 1.5y(k+2) + 7y(k+1) - 3.2y(k) = 23u(k+2) - 5u(k+1) + 0.5u(k) \quad (7)$$

- A state space model can now be derived using the companion matrix method exactly as for differential equations

Ordinary Difference Equations

Example

- Let $n = 3$, $m = 2$ and consider the difference equation:

$$y(k+3) + 1.5y(k+2) + 7y(k+1) - 3.2y(k) = 23u(k+2) - 5u(k+1) + 0.5u(k) \quad (7)$$

- A state space model can now be derived using the companion matrix method exactly as for differential equations

Ordinary Difference Equations

Example

Ordinary Difference Equations

Example

- The homgenerous version of the difference equation (7) can be converted as follows:

Ordinary Difference Equations

Example

- The homgenerous version of the difference equation (7) can be converted as follows:
- First put all but the highest order y's on the right side: $y(k+3) = 3.2y(k) - 7y(k+1) - 1.5(k+2)$

Ordinary Difference Equations

Example

- The homgenerous version of the difference equation (7) can be converted as follows:
- First put all but the highest order y's on the right side:
  $y(k+3) = 3.2y(k) - 7y(k+1) - 1.5(k+2)$
- Define the x's:
  $x_1(k) = y(k), x_2(k) = y(k+1), x_3(k) = y(k+2)$

Ordinary Difference Equations

Example

- The homgenerous version of the difference equation (7) can be converted as follows:
- First put all but the highest order y's on the right side:
  $y(k+3) = 3.2y(k) - 7y(k+1) - 1.5(k+2)$
- Define the x's:
  $x_1(k) = y(k), x_2(k) = y(k+1), x_3(k) = y(k+2)$
- Then the state space model is:

$$\begin{bmatrix} x_1(k+1) \\ x_2(k+1) \\ x_3(k+1) \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 3.2 & -7 & -1.5 \end{bmatrix} \begin{bmatrix} x_1(k) \\ x_2(k) \\ x_3(k) \end{bmatrix}$$

Ordinary Difference Equations

Example

- The homgenerous version of the difference equation (7) can be converted as follows:
- First put all but the highest order y's on the right side:
  $y(k+3) = 3.2y(k) - 7y(k+1) - 1.5(k+2)$
- Define the x's:
  $x_1(k) = y(k), x_2(k) = y(k+1), x_3(k) = y(k+2)$
- Then the state space model is:

$$\begin{bmatrix} x_1(k+1) \\ x_2(k+1) \\ x_3(k+1) \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 3.2 & -7 & -1.5 \end{bmatrix} \begin{bmatrix} x_1(k) \\ x_2(k) \\ x_3(k) \end{bmatrix}$$

The insertion of the u's as a matrix-vector product is left as an exercise.

Ordinary Difference Equations

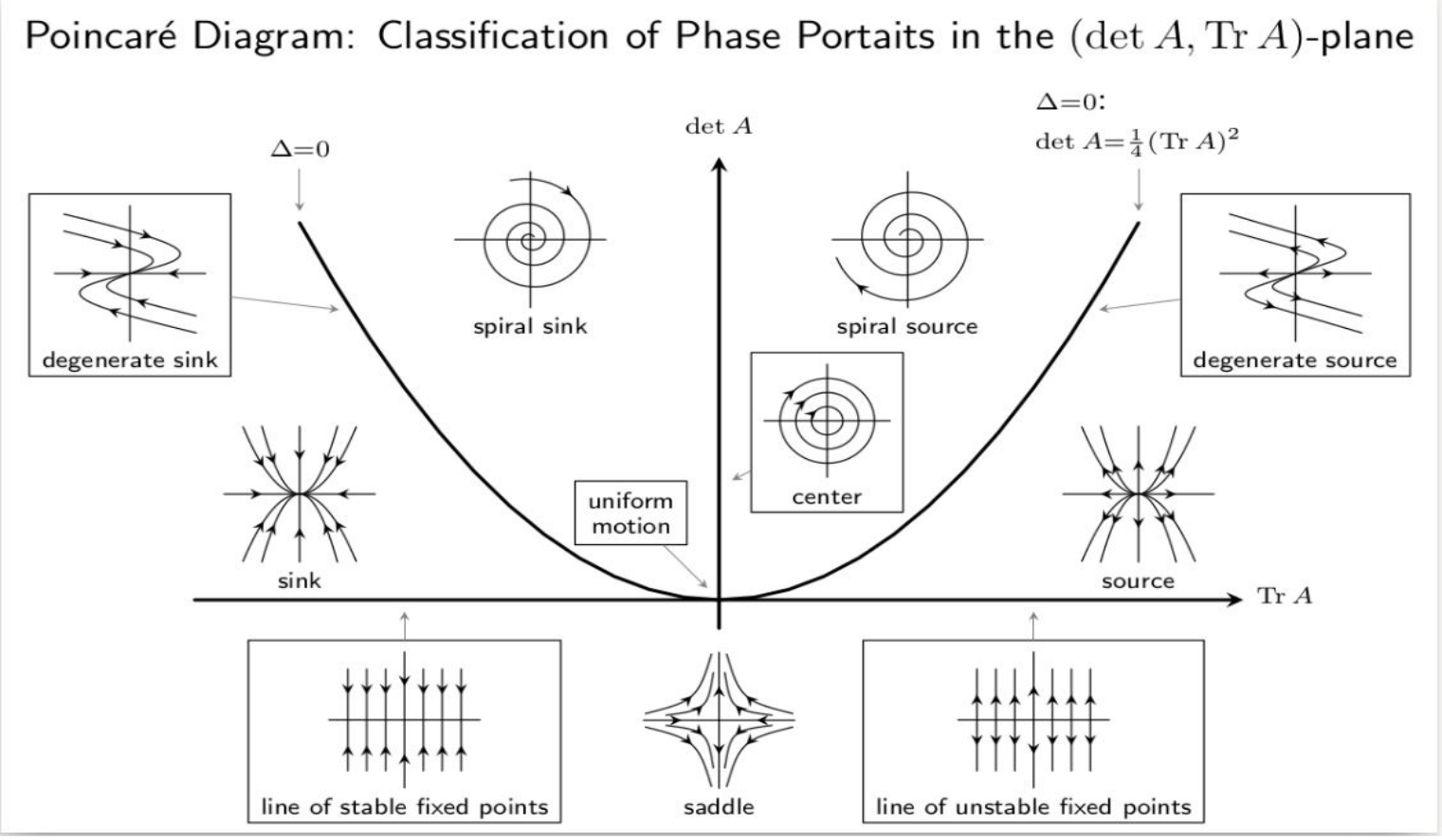
Example

- The homgenerous version of the difference equation (7) can be converted as follows:
- First put all but the highest order y's on the right side:
  $y(k+3) = 3.2y(k) - 7y(k+1) - 1.5(k+2)$
- Define the x's:
  $x_1(k) = y(k), x_2(k) = y(k+1), x_3(k) = y(k+2)$
- Then the state space model is:

$$\begin{bmatrix} x_1(k+1) \\ x_2(k+1) \\ x_3(k+1) \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 3.2 & -7 & -1.5 \end{bmatrix} \begin{bmatrix} x_1(k) \\ x_2(k) \\ x_3(k) \end{bmatrix}$$

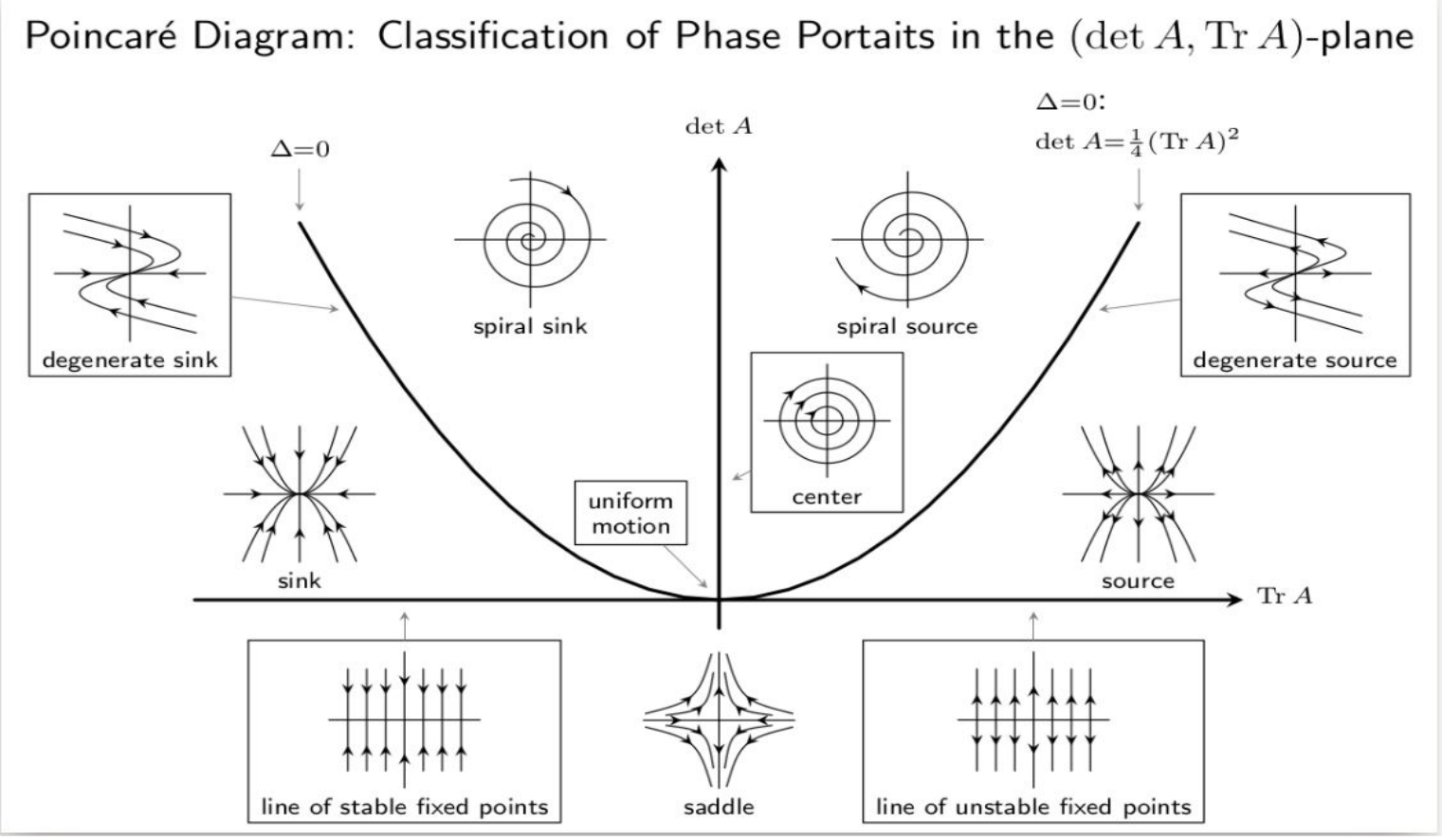
The insertion of the u's as a matrix-vector product is left as an exercise.

Stability



Source: By Freesodas - Gimp, CC BY-SA 4.0, https://commons.wikimedia.org/w/index.php?curid=67990034

Stability



Source: By Freesodas - Gimp, CC BY-SA 4.0, https://commons.wikimedia.org/w/index.php?curid=67990034

Software Tools

MATLAB toolboxes and Simulink blocks

Software Tools

MATLAB toolboxes and Simulink blocks

- MATLAB Control System Toolbox

- MATLAB Control System Toolbox
  - Linear System Models: LTI models as either transfer function or state space representation

Software Tools

MATLAB toolboxes and Simulink blocks

- MATLAB Control System Toolbox
  - Linear System Models: LTI models as either transfer function or state space representation
  - Functions for interconnecting models in series or parallel

- MATLAB Control System Toolbox
  - Linear System Models: LTI models as either transfer function or state space representation
  - Functions for interconnecting models in series or parallel
  - Functions for roundtrip converting from continuous-time to discrete-time

- MATLAB Control System Toolbox
  - Linear System Models: LTI models as either transfer function or state space representation
  - Functions for interconnecting models in series or parallel
  - Functions for roundtrip converting from continuous-time to discrete-time
  - Functions for scaling state space models

Software Tools

MATLAB toolboxes and Simulink blocks

- MATLAB Control System Toolbox
  - Linear System Models: LTI models as either transfer function or state space representation
  - Functions for interconnecting models in series or parallel
  - Functions for roundtrip converting from continuous-time to discrete-time
  - Functions for scaling state space models
- Simulink: Intuitive Block Diagram GUI Drag and Drop approach (see later slides)

Software Tools

MATLAB toolboxes and Simulink blocks

- MATLAB Control System Toolbox
  - Linear System Models: LTI models as either transfer function or state space representation
  - Functions for interconnecting models in series or parallel
  - Functions for roundtrip converting from continuous-time to discrete-time
  - Functions for scaling state space models
- Simulink: Intuitive Block Diagram GUI Drag and Drop approach (see later slides)

State Space Representation: Summary

State Space Representation: Summary

- State variables sometimes provided by 0th through $n - 1^{\text{st}}$ derivatives of single n^{th}-order differential equation plus internal dynamic coupling

State Space Representation: Summary

- State variables sometimes provided by 0th through $n - 1^{\text{st}}$ derivatives of single n^{th}-order differential equation plus internal dynamic coupling
- Equivalently, outputs of n integrators of system

State Space Representation: Summary

- State variables sometimes provided by 0th through $n - 1^{\text{st}}$ derivatives of single n^{th}-order differential equation plus internal dynamic coupling
- Equivalently, outputs of n integrators of system
- They represent implicit or explicit internal coupling of the system

State Space Representation: Summary

- State variables sometimes provided by 0th through $n - 1^{\text{st}}$ derivatives of single n^{th}-order differential equation plus internal dynamic coupling
- Equivalently, outputs of n integrators of system
- They represent implicit or explicit internal coupling of the system
- In both natural and artificial systems, state space representations are often derived directly from observed or designed couplings

State Space Representation: Summary

- State variables sometimes provided by 0th through $n - 1^{\text{st}}$ derivatives of single n^{th}-order differential equation plus internal dynamic coupling
- Equivalently, outputs of n integrators of system
- They represent implicit or explicit internal coupling of the system
- In both natural and artificial systems, state space representations are often derived directly from observed or designed couplings
- MATLAB Control Systems Toolbox and MATLAB / Simulink have many functions and blocks facilitating state space components and interconnections

State Space Representation: Summary

- State variables sometimes provided by 0th through $n - 1^{\text{st}}$ derivatives of single n^{th}-order differential equation plus internal dynamic coupling
- Equivalently, outputs of n integrators of system
- They represent implicit or explicit internal coupling of the system
- In both natural and artificial systems, state space representations are often derived directly from observed or designed couplings
- MATLAB Control Systems Toolbox and MATLAB / Simulink have many functions and blocks facilitating state space components and interconnections
- Stability an important topic but beyond scope of this course

State Space Representation: Summary

- State variables sometimes provided by 0th through $n - 1^{\text{st}}$ derivatives of single n^{th}-order differential equation plus internal dynamic coupling
- Equivalently, outputs of n integrators of system
- They represent implicit or explicit internal coupling of the system
- In both natural and artificial systems, state space representations are often derived directly from observed or designed couplings
- MATLAB Control Systems Toolbox and MATLAB / Simulink have many functions and blocks facilitating state space components and interconnections
- Stability an important topic but beyond scope of this course

Appendix: State Space Representation

Matlab / Simulink Example

Appendix: State Space Representation

Matlab / Simulink Example

- Four parameters A, B, C, D from Matlab workspace

Appendix: State Space Representation

Matlab / Simulink Example

- Four parameters A, B, C, D from Matlab workspace
- Measurement model added external to Simulink's state space block

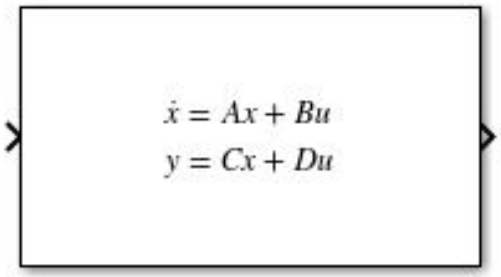
Appendix: State Space Representation

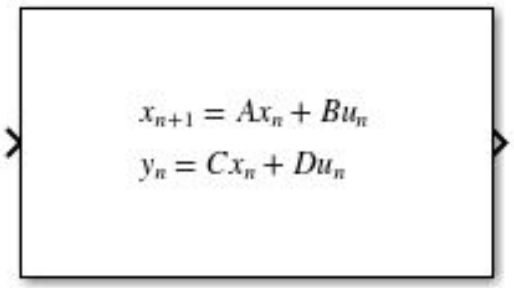
Matlab / Simulink Example

- Four parameters A, B, C, D from Matlab workspace
- Measurement model added external to Simulink's state space block

Appendix: State Space Representation

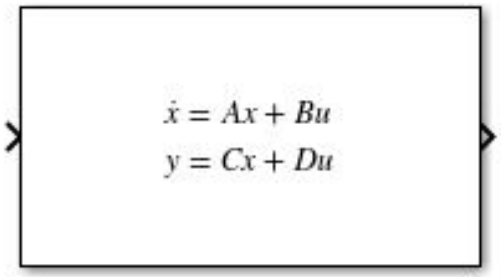
Matlab / Simulink Example

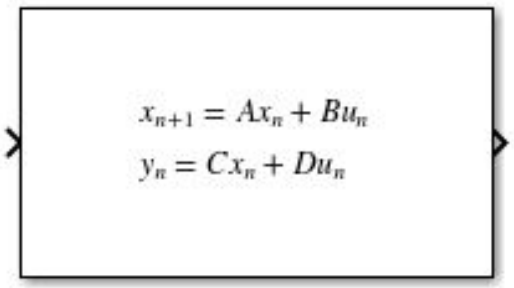




Appendix: State Space Representation

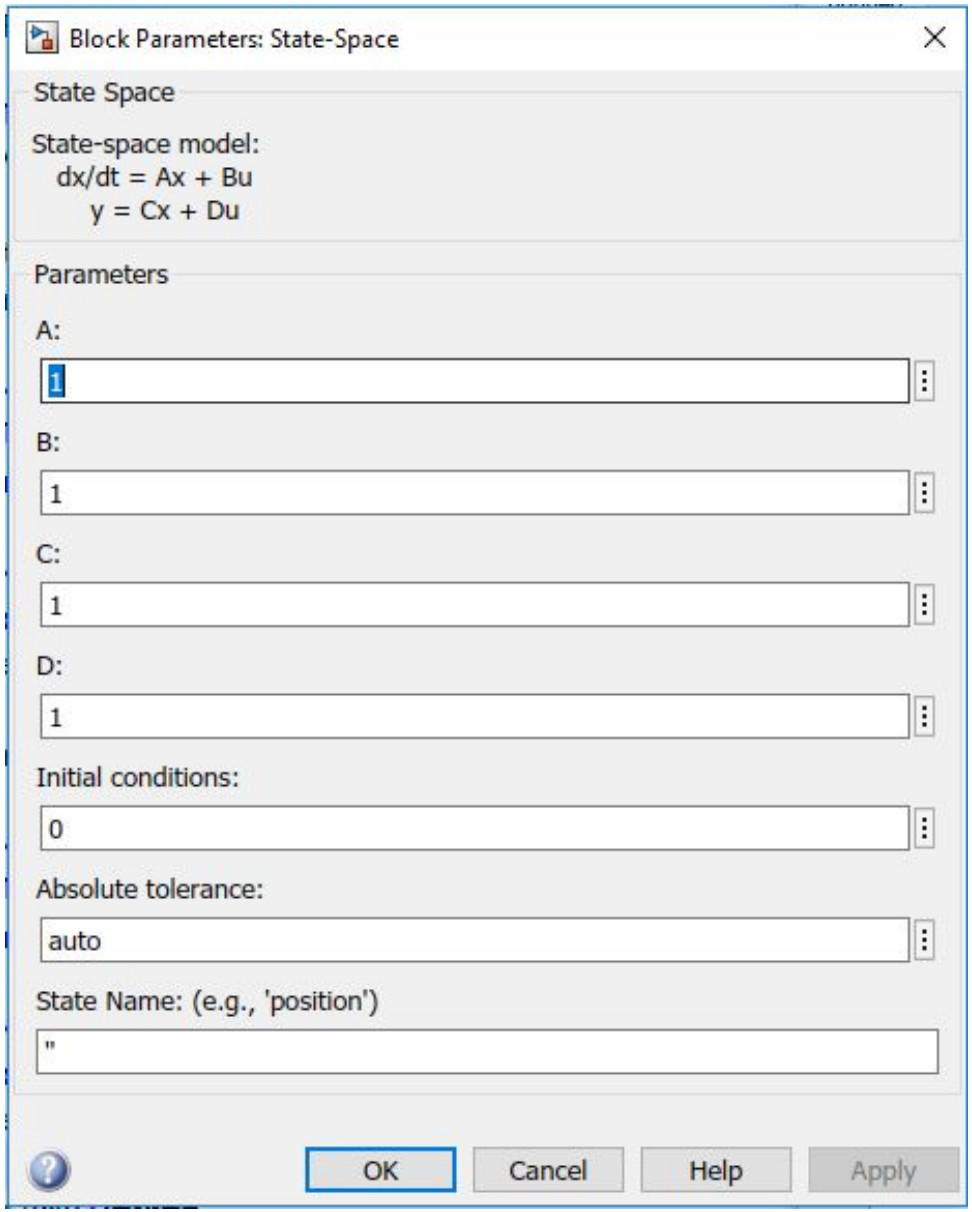
Matlab / Simulink Example





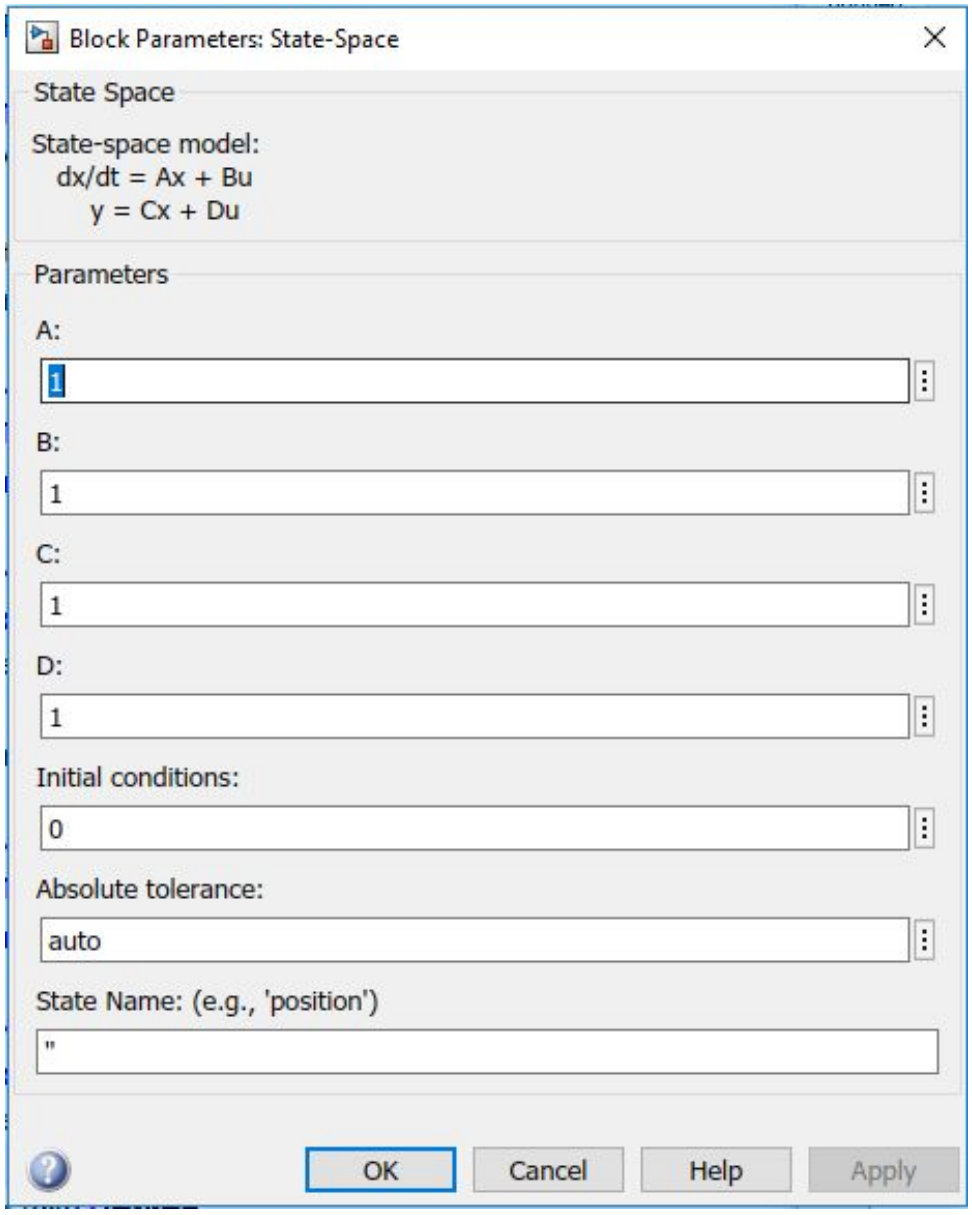
Appendix: State Space Representation

Matlab / Simulink Example



Appendix: State Space Representation

Matlab / Simulink Example



Summary

Vector Check

Summary

Vector Check

- Where are we?

Summary

Vector Check

- Where are we?
  - TBD

Summary

Vector Check

- Where are we?
  - TBD
  - Reminder: The python scripts are in `https://github.com/musicarroll/kalman_course/src`

Summary

Vector Check

- Where are we?
  - TBD
  - Reminder: The python scripts are in `https://github.com/musicarroll/kalman_course/src`

Summary

Vector Check

- Where are we?
  - TBD
  - Reminder: The python scripts are in `https://github.com/musicarroll/kalman_course/src`
- What's next?

Summary

Vector Check

- Where are we?
  - TBD
  - Reminder: The python scripts are in `https://github.com/musicarroll/kalman_course/src`
- What's next?
  - TBD

Summary

Vector Check

- Where are we?
  - TBD
  - Reminder: The python scripts are in `https://github.com/musicarroll/kalman_course/src`
- What's next?
  - TBD